

Tugas 11: Tugas Praktikum Mandiri 11 – DBSCAN CLUSTERING

Analisis Sebaran Airbnb New York City Menggunakan Koordinat GPS

Revani – 0110224111 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224111@student.nurulfikri.ac.id

Abstract. Pada tugas latihan praktikum 11 ini saya mencoba memahami bagaimana algoritma DBSCAN bekerja dalam mengelompokkan data berdasarkan kepadatannya. Saya menggunakan dataset Airbnb New York City karena dataset ini punya ribuan titik koordinat GPS sehingga cocok banget untuk melihat bagaimana DBSCAN membentuk cluster di daerah yang padat listing dan memisahkan daerah yang sepi sebagai noise. Dalam analisis ini saya melakukan pembersihan data, visualisasi heatmap lokasi Airbnb, menentukan nilai eps dan min_samples memakai grafik K-Distance, lalu melakukan clustering dan memvisualisasikan hasilnya baik lewat PCA maupun peta GPS. Dari praktikum ini saya belajar bagaimana pola keramaian Airbnb di New York terbentuk dan bagaimana DBSCAN lebih unggul dibanding algoritma clustering yang harus menentukan jumlah cluster di awal seperti K-Means. Praktikum ini membantu saya memahami penerapan algoritma clustering pada data spasial dan cara membaca pola persebaran geografis.

1. Penjelasan Hasil Latihan Praktikum

Pada awal praktikum, saya memuat dataset Airbnb New York yang berisi informasi seperti nama listing, harga, wilayah, latitude, longitude, dan kolom lainnya. Fokus utama saya ada pada kolom latitude dan longitude karena DBSCAN akan dipakai pada data GPS.

Setelah mengecek jumlah baris dan kolom, saya melakukan cleaning data dengan mengecek missing value dan mengisi kolom seperti reviews per month yang kosong dengan nol. Setelah itu, saya buat visualisasi heatmap menggunakan folium untuk melihat daerah mana yang paling padat listing-nya. Dari heatmap terlihat bahwa Manhattan jadi daerah yang paling merah, artinya listing di sana sangat padat.

Lanjut setelah visualisasi awal, saya mengkonversi data GPS ke GeoDataFrame supaya bisa digambar ke peta asli menggunakan basemap dari OpenStreetMap. Hasilnya benar-benar memperjelas sebaran listing. Setelah itu, saya siapkan data numerik yang akan dipakai untuk clustering dan melakukan normalisasi menggunakan RobustScaler supaya data yang punya outlier (contohnya harga Airbnb yang ekstrem) tidak membuat scaling jadi rusak.

Setelah scaling, saya harus menentukan nilai eps dan min_samples karena itu dua parameter wajib DBSCAN. Untuk eps saya gunakan grafik K-Distance. Grafik itu memperlihatkan jarak tetangga ke-10 dari setiap data (karena min_samples saya tetapkan 10). Titik yang "tekukannya" paling jelas saya pakai sebagai eps, yaitu sekitar 0.30.

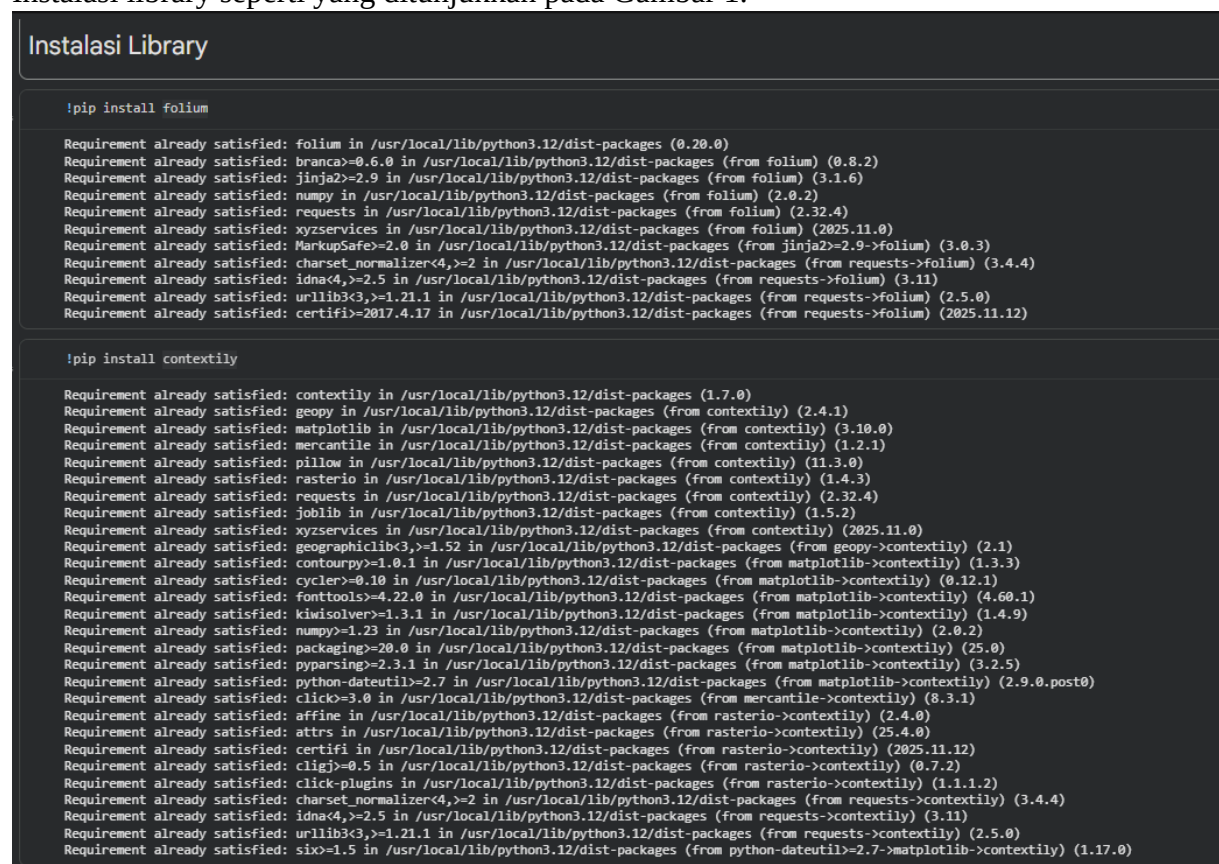
Setelah itu saya jalankan DBSCAN dan hasilnya menghasilkan beberapa cluster serta ratusan noise. Noise di sini adalah data yang tidak cukup dekat dengan cluster manapun, biasanya listing yang lokasinya terlalu jauh atau terpencil. Lalu saya visualisasikan hasil clustering memakai PCA 2 dimensi dan juga dipetakan ke GPS sehingga terlihat jelas cluster berkumpul di Manhattan dan Brooklyn. Hasil akhirnya menunjukkan bahwa DBSCAN bisa menangkap pola kepadatan Airbnb dengan sangat baik.

2. Penjelasan Kode Program

a. Instalasi Library

```
!pip install folium
!pip install contextily
```

Instalasi library seperti yang ditunjukkan pada Gambar 1.



```
Instalasi Library

!pip install folium

Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2025.11.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.9->folium) (3.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2025.11.12)

!pip install contextily

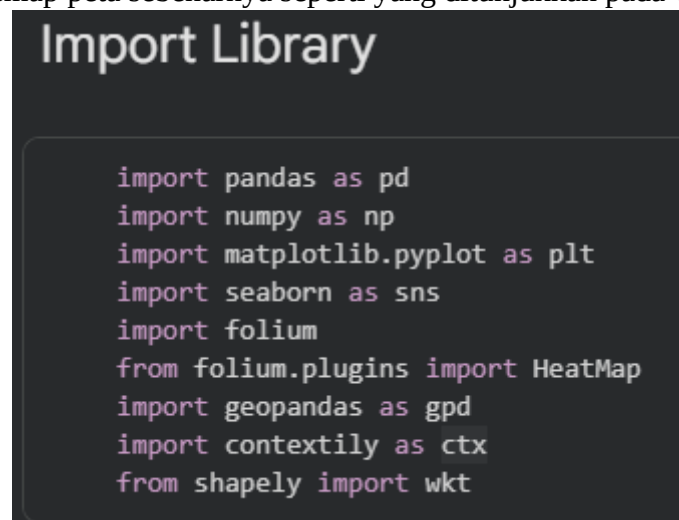
Requirement already satisfied: contextily in /usr/local/lib/python3.12/dist-packages (1.7.0)
Requirement already satisfied: geoppy in /usr/local/lib/python3.12/dist-packages (from contextily) (2.4.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from contextily) (3.10.0)
Requirement already satisfied: mercantile in /usr/local/lib/python3.12/dist-packages (from contextily) (1.2.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from contextily) (11.3.0)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (from contextily) (1.4.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from contextily) (2.32.4)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from contextily) (1.5.2)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from contextily) (2025.11.0)
Requirement already satisfied: geographiclib<3,>=1.52 in /usr/local/lib/python3.12/dist-packages (from geoppy->contextily) (2.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (25.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->contextily) (2.9.0.post0)
Requirement already satisfied: click>=3.0 in /usr/local/lib/python3.12/dist-packages (from mercantile->contextily) (8.3.1)
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-packages (from rasterio->contextily) (2.4.0)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages (from rasterio->contextily) (25.4.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from rasterio->contextily) (2025.11.12)
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.12/dist-packages (from rasterio->contextily) (0.7.2)
Requirement already satisfied: click-plugins in /usr/local/lib/python3.12/dist-packages (from rasterio->contextily) (1.1.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->contextily) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->contextily) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->contextily) (2.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->contextily) (1.17.0)
```

Gambar 1. Pada bagian ini ditunjukkan instalasi library.

b. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import HeatMap
import geopandas as gpd
import contextily as ctx
from shapely import wkt
```

Bagian ini untuk memanggil semua library yang akan dipake. Dengan library pandas buat ngolah data tabel, numpy untuk operasi numerik, matplotlib atau seaborn untuk grafik, folium buat peta interaktif, geopandas biar latitude longitude bisa jadi titik koordinat, dan contextily buat nambahin basemap peta sebenarnya seperti yang ditunjukkan pada Gambar 2.



Gambar 2. Pada bagian ini ditunjukkan import library.

c. Load Dataset

```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Machine
Learning/Praktikum11/Data/AB_NYC_2019.csv')
df.head()
```

Kode ini buat baca dataset Airbnb lalu saya tampilkan 5 baris pertama. Di sini saya ngecek apakah data sudah masuk atau belum seperti yang ditunjukkan pada Gambar 3.

Load Dataset

```
df = pd.read_csv("../content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum11/Data/AB_NYC_2019.csv")
df.head()
```

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calculated_host_listings_count	availability_365
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.64749	-73.97237	Private room	149	1	9	2016-10-19	0.21		6
1	2595	Skyline Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	1	45	2019-05-21	0.38		2
2	3647	THE VILLAGE OF HARLEM - NEW YORK I	4632	Elisabeth	Manhattan	Harlem	40.80902	-73.94190	Private room	150	3	0	NaN	NaN		1
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	-73.95976	Entire home/apt	89	1	270	2019-07-05	4.64		1
4	5022	Entire Apt. Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.94399	Entire home/apt	80	10	9	2018-11-19	0.10		1

Gambar 3. Pada bagian ini ditunjukkan load dataset.

d. Informasi Dataset

```
df.info()
```

Outputnya nunjukkin ada 48 ribuan baris dan beberapa kolom seperti name, neighbourhood, latitude, longitude, price, dll. Tipe datanya juga keliatan seperti yang ditunjukkan pada Gambar 4.

Informasi Dataset

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   id                                     48895 non-null  int64
 1   name                                  48879 non-null  object
 2   host_id                               48895 non-null  int64
 3   host_name                             48874 non-null  object
 4   neighbourhood_group                   48895 non-null  object
 5   neighbourhood                         48895 non-null  object
 6   latitude                             48895 non-null  float64
 7   longitude                             48895 non-null  float64
 8   room_type                             48895 non-null  object
 9   price                                 48895 non-null  int64
10  minimum_nights                        48895 non-null  int64
11  number_of_reviews                     48895 non-null  int64
12  last_review                           38843 non-null  object
13  reviews_per_month                     38843 non-null  float64
14  calculated_host_listings_count         48895 non-null  int64
15  availability_365                       48895 non-null  int64
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB
```

Gambar 4. Pada bagian ini ditunjukkan cek informasi kolom.

d. Cek Missing Value

```
df.isnull().sum()
```

Bagian kode ini dipakai buat ngecek berapa banyak data yang kosong (NaN) di tiap kolom. Outputnya nunjukkin jumlah nilai kosong per kolom dengan last_review punya 10052 nilai kosong, reviews_per_month juga 10052 kosong, host_name ada 21 kosong, Kolom lain seperti id, name, latitude, longitude, dll tidak ada yang kosong. Tujuannya supaya kita tahu kolom mana yang perlu dibersihkan seperti yang ditunjukkan pada Gambar 5.

Cleaning Data

[75] `df.isnull().sum()`

...	0
id	0
name	16
host_id	0
host_name	21
neighbourhood_group	0
neighbourhood	0
latitude	0
longitude	0
room_type	0
price	0
minimum_nights	0
number_of_reviews	0
last_review	10052
reviews_per_month	10052
calculated_host_listings_count	0
availability_365	0

dtype: int64

Gambar 5. Pada bagian ini ditunjukkan cek missing value.

e. Cek duplikat, mengisi missing value, menghapus baris

```
df.duplicated().sum()
```

Kode ini dipakai untuk mengecek berapa banyak baris duplikat di dataset. Kalau hasilnya 0, berarti tidak ada baris yang sama persis dan dataset aman dari duplikasi. Duplikasi penting dicek karena kalau ada baris kembar, analisis bisa jadi bias.

```
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
```

Bagian ini mengisi kolom reviews_per_month yang kosong.

```
df.dropna(inplace=True)
```

Bagian ini menghapus seluruh baris yang masih punya missing value seperti yang ditunjukkan pada Gambar 6.

```
df.duplicated().sum()

np.int64(0)

df['reviews_per_month'] = df['reviews_per_month'].fillna(0)

# drop fitur lain yang kosong
df.dropna(inplace=True)
```

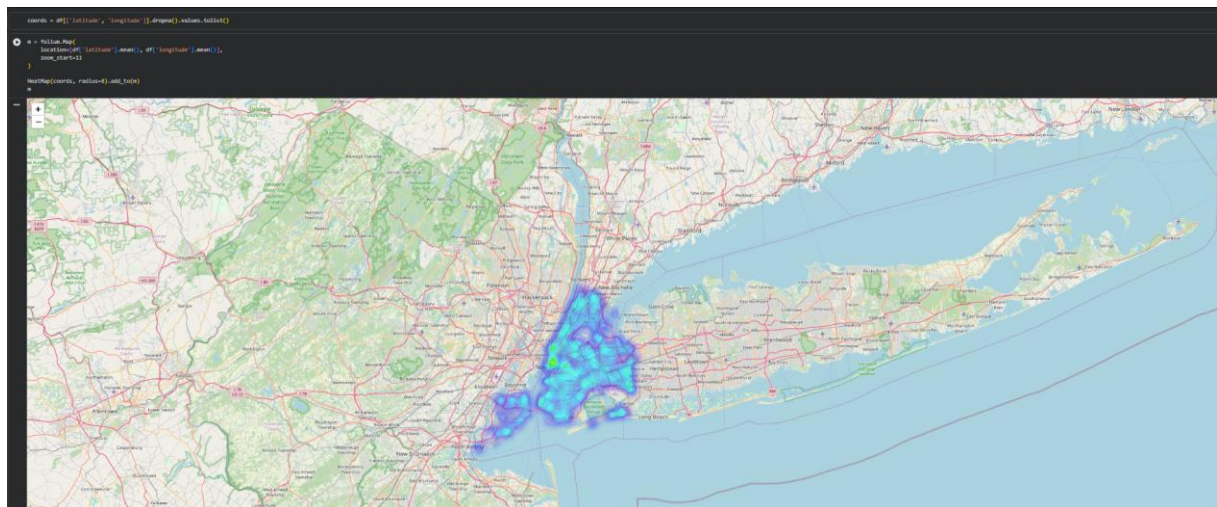
Gambar 6. Pada bagian ini ditunjukkan cek duplikat, isi missing value, menghapus baris.

f. Visualisasi Heatmap Sebaran NYC Airbnb (Latitude-Longitude)

```
coords = df[['latitude', 'longitude']].dropna().values.tolist()
m = folium.Map(
    location=[df['latitude'].mean(), df['longitude'].mean()],
    zoom_start=11
)

HeatMap(coords, radius=8).add_to(m)
m
```

Outputnya berupa peta interaktif dengan warna merah menunjukkan daerah yang padat Airbnb. Hasilnya jelas Manhattan paling merah seperti yang ditunjukkan pada Gambar 7.



Gambar 7. Pada bagian ini ditunjukkan Heatmap GPS.

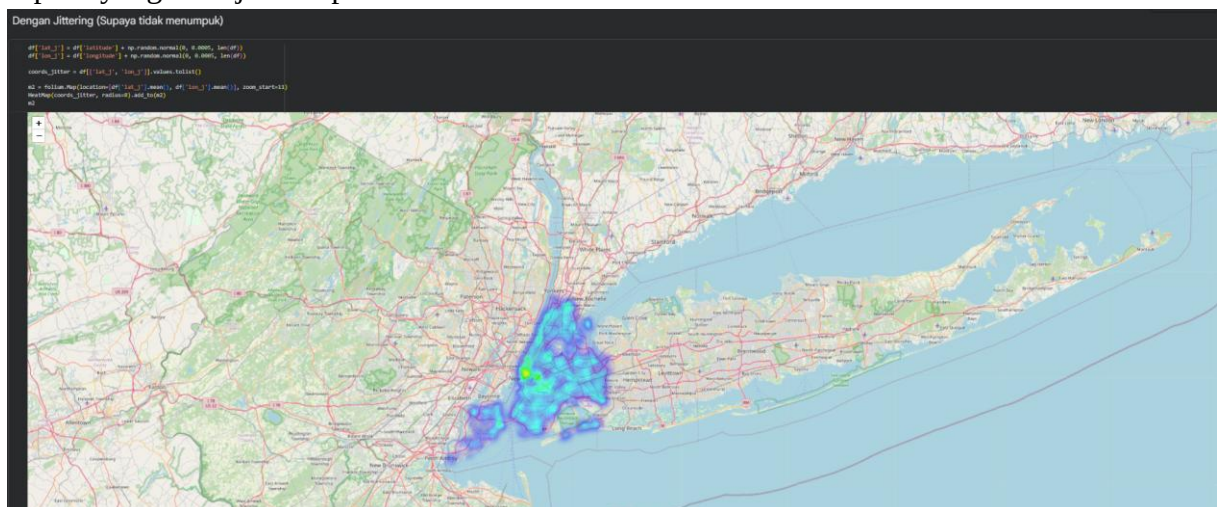
g. Dengan Jittering

```
df['lat_j'] = df['latitude'] + np.random.normal(0, 0.0005, len(df))
df['lon_j'] = df['longitude'] + np.random.normal(0, 0.0005, len(df))

coords_jitter = df[['lat_j', 'lon_j']].values.tolist()

m2 = folium.Map(location=[df['lat_j'].mean(), df['lon_j'].mean()],
zoom_start=11)
HeatMap(coords_jitter, radius=8).add_to(m2)
m2
```

Jittering menggeser setiap titik sedikit sehingga tumpukan titik jadi terlihat sebagai area keramaian. Ini berguna untuk heatmap, scatter, atau peta titik supaya kepadatan lebih terlihat seperti yang ditunjukkan pada Gambar 8.



Gambar 8. Pada bagian ini ditunjukkan Jittering.

h. Visualisasi Basemap NYC

```
gdf = gpd.GeoDataFrame(  
    df.dropna(subset=['latitude', 'longitude']),  
    geometry=gpd.points_from_xy(df['longitude'], df['latitude']),  
    crs='EPSG:4326'  
)  
  
gdf = gdf.to_crs(epsg=3857)
```

Kode ini untuk membuat GeoDataFrame dengan kolom geometry berupa titik (lon, lat). Serta melakukan konversi ke web mercator (meter) supaya contextily bisa menempelkan tile peta.

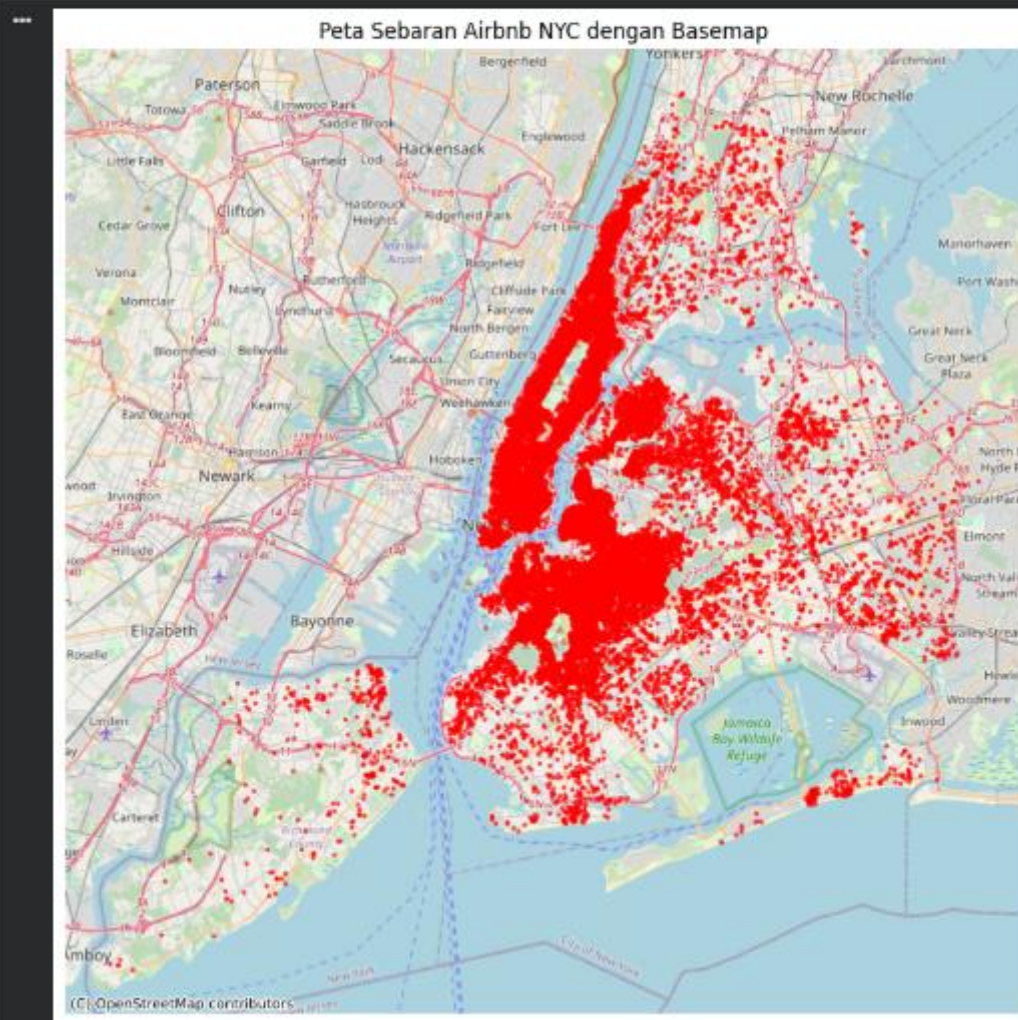
```
fig, ax = plt.subplots(figsize=(10, 10))  
gdf.plot(ax=ax, alpha=0.6, markersize=3, color='red')  
  
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)  
ax.set_axis_off()  
plt.title("Peta Sebaran Airbnb NYC dengan Basemap")  
plt.show()
```

Kode ini untuk membuat plot titik-titik ke axes matplotlib. Output yang dihasilkan yaitu peta statis berbasemap dengan titik merah representasi listing seperti yang ditunjukkan pada Gambar 9.

Visualisasi Basemap NYC

```
gdf = gpd.GeoDataFrame(  
    df.dropna(subset=['latitude', 'longitude']),  
    geometry=gpd.points_from_xy(df['longitude'], df['latitude']),  
    crs="EPSG:4326"  
)  
  
gdf = gdf.to_crs(epsg=3857)
```

```
fig, ax = plt.subplots(figsize=(10, 10))  
gdf.plot(ax=ax, alpha=0.6, markersize=3, color='red')  
  
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)  
ax.set_axis_off()  
plt.title("Peta Sebaran Airbnb NYC dengan Basemap")  
plt.show()
```



Gambar 9. Pada bagian ini ditunjukkan Visualisasi Basemap.

i. Visualisasi Peta dan Grafik

```
fig, ax = plt.subplots(figsize=(12, 10))
```

```
gdf.plot(  
    ax=ax,  
    column='price',  
    cmap='OrRd',  
    markersize=5,  
    legend=True  
)  
  
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)  
ax.set_title("Peta Sebaran Harga Airbnb (NYC)")  
ax.set_axis_off()  
plt.show()
```

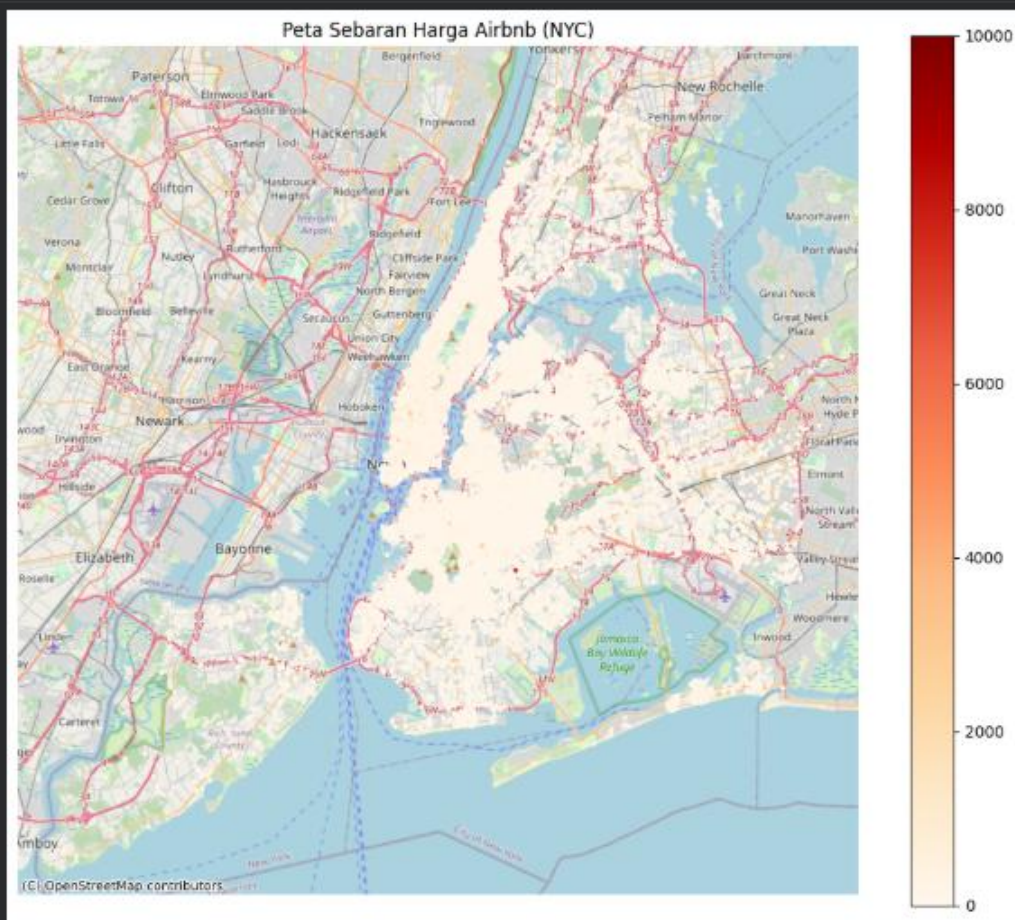
Kode untuk menampilkan titik yang dibaeri warna sesuai price. Ini membantu melihat area dengan listing yang relative mahal. Output yang dihasilkan adalah peta berwarna dan legenda harga seperti yang ditunjukkan pada Gambar 10.

Visualisasi Peta + Grafik Harga Tertinggi & Terendah

```
fig, ax = plt.subplots(figsize=(12, 10))

gdf.plot(
    ax=ax,
    column='price',
    cmap='OrRd',
    markersize=5,
    legend=True
)

ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)
ax.set_title("Peta Sebaran Harga Airbnb (NYC)")
ax.set_axis_off()
plt.show()
```

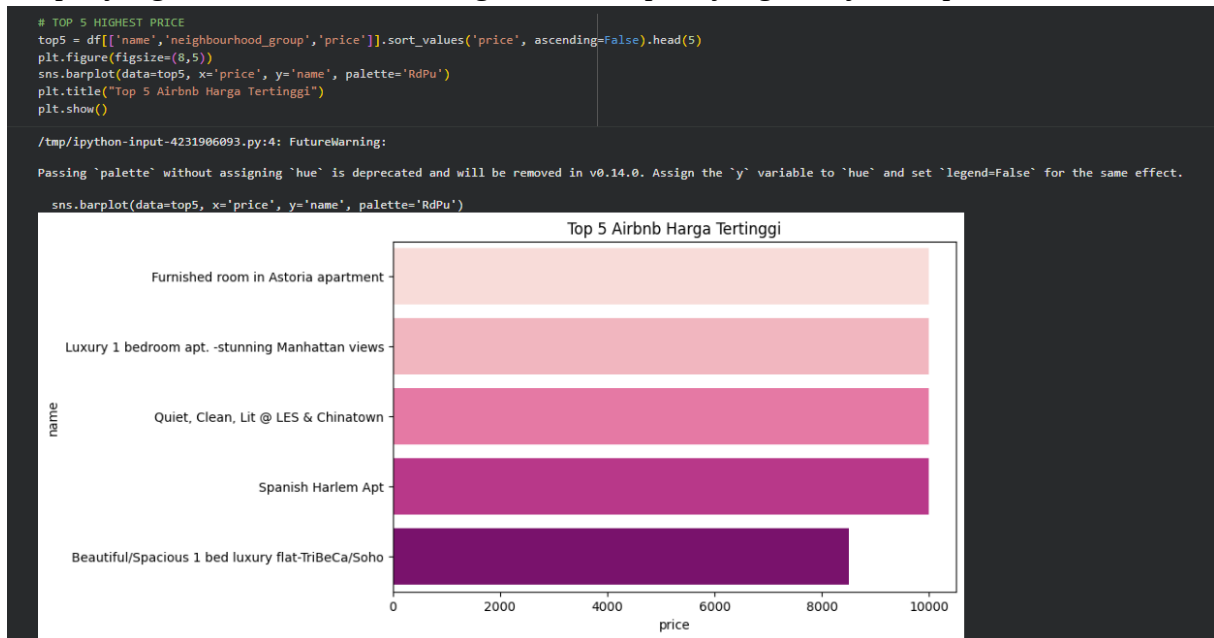


Gambar 10. Pada bagian ini ditunjukkan Visualisasi Grafik dan Peta.

j. Harga Tertinggi

```
top5 = df[['name', 'neighbourhood_group', 'price']].sort_values('price',
    ascending=False).head(5)
plt.figure(figsize=(8,5))
sns.barplot(data=top5, x='price', y='name', palette='RdPu')
plt.title("Top 5 Airbnb Harga Tertinggi")
plt.show()
```

Kode ini untuk memilih 5 listing termahal lalu membuat barplot supaya nama listing terbaca. Output yang dihasilkan chart 5 listing termahal seperti yang ditunjukkan pada Gambar 11.



Gambar 11. Pada bagian ini ditunjukkan harga tertinggi.

j. Harga Terendah

```
bottom5 = df[df['price'] > 0][['name', 'neighbourhood_group', 'price']] \
    .sort_values('price', ascending=True).head(5)
plt.figure(figsize=(8,5))
sns.barplot(data=bottom5, x='price', y='name', palette='Blues')
plt.title("Top 5 Airbnb Harga Terendah (Price > 0)")
plt.show()
```

Kode ini untuk memilih 5 listing termurah lalu membuat barplot supaya nama listing terbaca. Output yang dihasilkan chart 5 listing termurah (price > 0) seperti yang ditunjukkan pada Gambar 12.



Gambar 12. Pada bagian ini ditunjukkan harga terendah.

k. Uji Normalitas

```
from scipy.stats import shapiro

cols = ['price', 'minimum_nights', 'number_of_reviews',
'reviews_per_month', 'availability_365']
normality_df = df[cols]

stat, p = shapiro(normality_df['price'])
print("Statistik:", stat)
print("P-value:", p)

if p > 0.05:
    print("Data berdistribusi normal (H0 diterima)")
else:
    print("Data tidak berdistribusi normal (H0 ditolak)")
```

Kode ini untuk menguji apakah distribusi price normal. Output yang dihasilkan adalah nilai stat dan p-value, plus teks hasil hipotesis seperti yang ditunjukkan pada Gambar 13.

```
Uji Normalitas (Shapiro)

from scipy.stats import shapiro

cols = ['price', 'minimum_nights', 'number_of_reviews', 'reviews_per_month', 'availability_365']
normality_df = df[cols]

stat, p = shapiro(normality_df['price'])
print("Statistik:", stat)
print("P-value:", p)

if p > 0.05:
    print("Data berdistribusi normal (H0 diterima)")
else:
    print("Data tidak berdistribusi normal (H0 ditolak)")

Statistik: 0.32336252101402607
P-value: 7.526189204179659e-144
Data tidak berdistribusi normal (H0 ditolak)
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: UserWarning: scipy.stats.shapiro: For N > 5000, computed p-value may not be accurate. Current N is 38821.
  res = hypotest_fun_out(*samples, **kwargs)
```

Gambar 13. Pada bagian ini ditunjukkan uji normalitas.

1. Korelasi Spearman

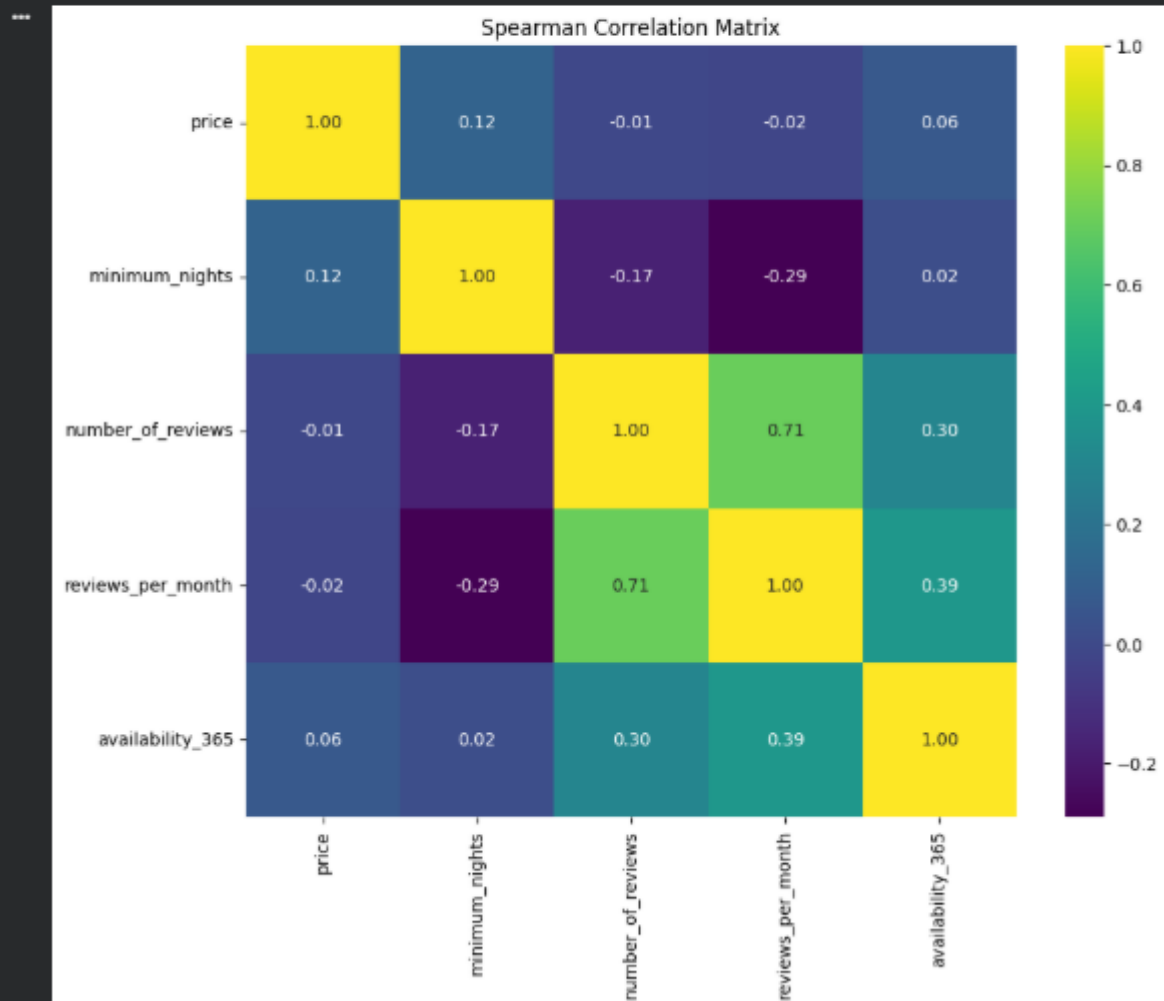
```
spearman_corr = normality_df.corr(method='spearman')

plt.figure(figsize=(10, 8))
sns.heatmap(spearman_corr, annot=True, cmap='viridis', fmt=".2f",
square=True)
plt.title("Spearman Correlation Matrix")
plt.show()
```

Spearman dipakai karena data tidak normal dan beberapa fitur ordinal/skewed. Heatmap menunjukkan hubungan monotonic antar fitur. Output yang ditampilkan adalah heatmap korelasi seperti yang ditunjukkan pada Gambar 14.

Korelasi Spearman

```
spearman_corr = normality_df.corr(method='spearman')  
  
plt.figure(figsize=(10, 8))  
sns.heatmap(spearman_corr, annot=True, cmap='viridis', fmt=".2f", square=True)  
plt.title("Spearman Correlation Matrix")  
plt.show()
```



Gambar 14. Pada bagian ini ditunjukkan korelasi spearman.

m. Normalisasi data dengan RobustScaler

```
from sklearn.preprocessing import RobustScaler  
  
cols_for_clustering = [  
    'price',  
    'minimum_nights',  
    'number_of_reviews',  
    'reviews_per_month',  
]
```



```

        'availability_365'
    ]

    X = df[cols_for_clustering]

    scaler = RobustScaler()
    X_scaled_array = scaler.fit_transform(X)

    X_scaled = pd.DataFrame(X_scaled_array, columns=cols_for_clustering,
                             index=X.index)
    X_scaled.head()

```

RobustScaler dipilih karena tahan terhadap outlier menggunakan median dan IQR, cocok untuk price yang sangat skew atau ada harga ekstrem. Hasilnya data berpusat dan sudah bisa dipakai untuk perhitungan jarak. Output yang ditampilkan adalah tabel 5 baris pertama dari data yang sudah diskalakan seperti yang ditunjukkan pada Gambar 15.

Normalisasi Data dengan RobustScaler

```

from sklearn.preprocessing import RobustScaler

cols_for_clustering = [
    'price',
    'minimum_nights',
    'number_of_reviews',
    'reviews_per_month',
    'availability_365'
]

X = df[cols_for_clustering]

scaler = RobustScaler()
X_scaled_array = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled_array, columns=cols_for_clustering, index=X.index)
X_scaled.head()

```

	price	minimum_nights	number_of_reviews	reviews_per_month	availability_365
0	0.475248	-0.333333	0.000000	-0.278689	1.353712
1	1.227723	-0.333333	1.200000	-0.185792	1.310044
3	-0.118812	-0.333333	8.700000	2.142077	0.606987
4	-0.207921	2.666667	0.000000	-0.338798	-0.240175
5	0.980198	0.333333	2.166667	-0.071038	0.323144

Gambar 15. Pada bagian ini ditunjukkan RobustScaler.

n. EPS dengan K-Distance Graph

```
from sklearn.neighbors import NearestNeighbors

min_samples = 10
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

distances = np.sort(distances[:, -1])

plt.figure(figsize=(8,5))
plt.plot(distances)
plt.title("K-Distance Graph (min_samples = 10)")
plt.xlabel("Data Points sorted by distance")
plt.ylabel("Distance")
plt.grid(True)
plt.show()

distances[-20:]
```

Saya menghitung jarak ke tetangga ke min_samples 10, dan menampilkan 20 jarak terbesar. Output yang dihasilkan adalah grafik K-distance menggunakan mata untuk kira-kira eps, mis 0.3 seperti yang ditunjukkan pada Gambar 16.

Menentukan EPS dengan K-Distance Graph

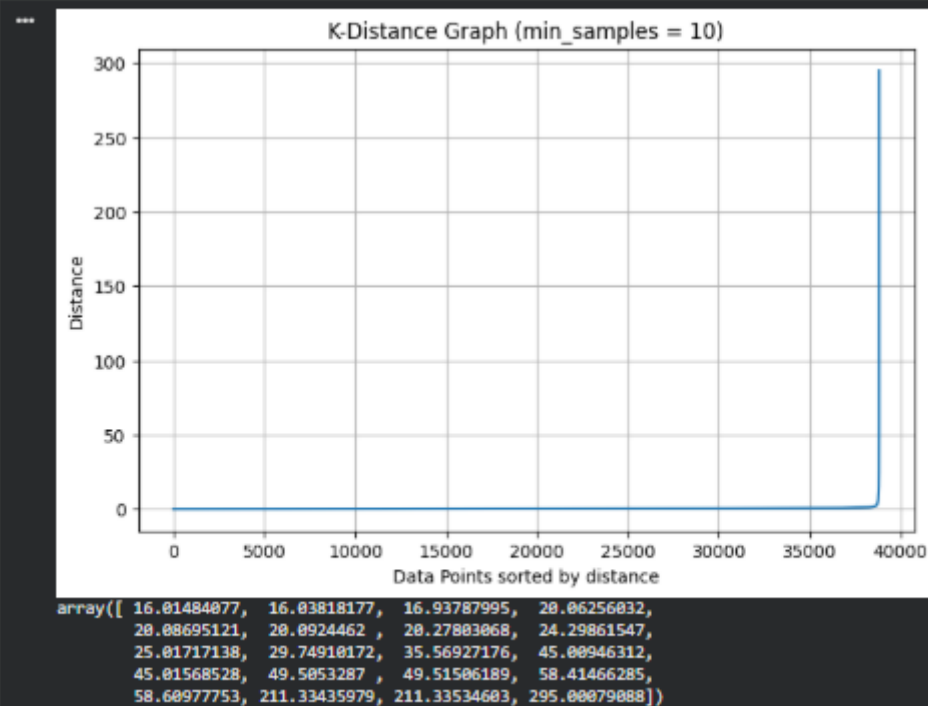
```
from sklearn.neighbors import NearestNeighbors

min_samples = 10
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

distances = np.sort(distances[:, -1])

plt.figure(figsize=(8,5))
plt.plot(distances)
plt.title("K-Distance Graph (min_samples = 10)")
plt.xlabel("Data Points sorted by distance")
plt.ylabel("Distance")
plt.grid(True)
plt.show()

distances[-20:]
```



Gambar 16. Pada bagian ini ditunjukkan EPS dengan K-Distance Graph.

o. DBSN Clustering

```
from sklearn.cluster import DBSCAN

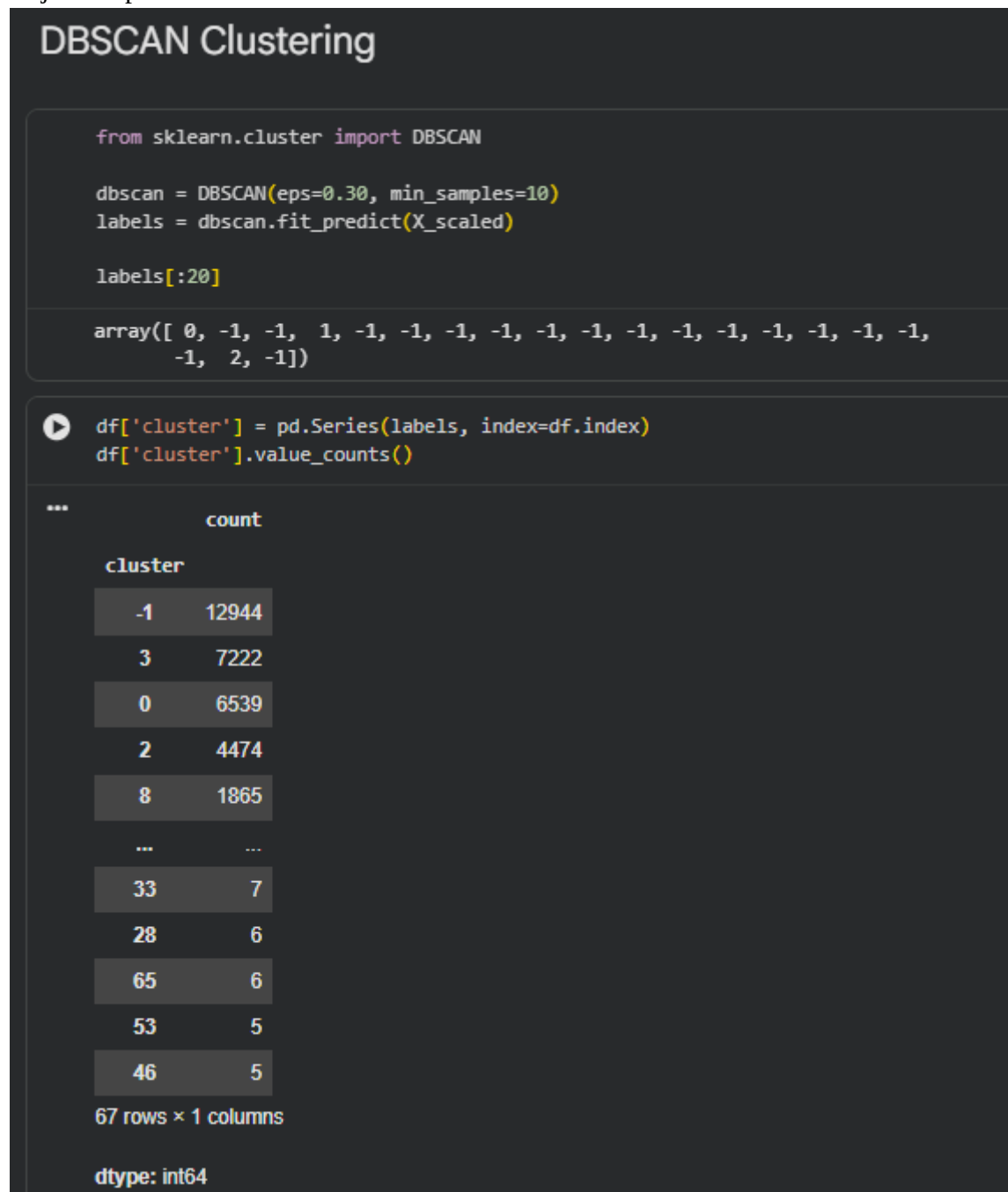
dbscan = DBSCAN(eps=0.30, min_samples=10)
labels = dbscan.fit_predict(X_scaled)

labels[:20]

df['cluster'] = pd.Series(labels, index=df.index)
```

```
df['cluster'].value_counts()
```

Kode ini untuk menjalankan clustering density-based, mengeluarkan label per baris, menampilkan 20 label pertama sebagai contoh dan menyimpan label untuk hitung distribusi per cluster. Outputnya yaitu distribusi jumlah titik tiap cluster dan jumlah noise seperti yang ditunjukkan pada Gambar 17.



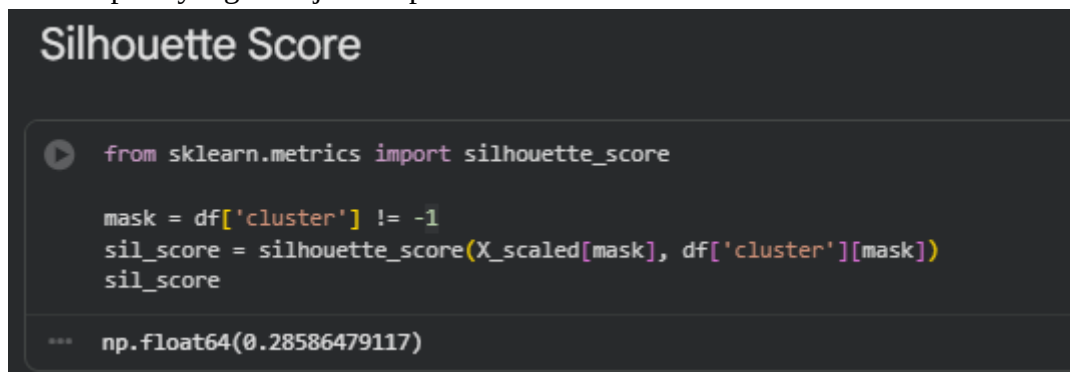
Gambar 17. Pada bagian ini ditunjukkan DBSCAN Clustering.

p. Silhouette Score

```
from sklearn.metrics import silhouette_score

mask = df['cluster'] != -1
sil_score = silhouette_score(X_scaled[mask], df['cluster'][mask])
sil_score
```

Kode ini hanya dapat dihitung untuk cluster bukan noise. Score derange yaitu semakin dekat ke 1 semakin baik pemisahan cluster. Untuk DBSCAN dan data spasial skor sering rendah atau tidak stabil bukan ukuran sempurna untuk DBSCAN tapi tetap referensi. Output yang dihasilkan nilai sil_score (float). Jika mask kosong (semua noise), fungsi error, jadi cek jumlah cluster > 1 seperti yang ditunjukkan pada Gambar 18.



Gambar 18. Pada bagian ini ditunjukkan Silhouette Score.

q. Visualisasi Clustering PCA 2D

```
from sklearn.decomposition import PCA

dbscan = DBSCAN(eps=0.6, min_samples=10)
labels = dbscan.fit_predict(X_scaled)

df['cluster'] = labels

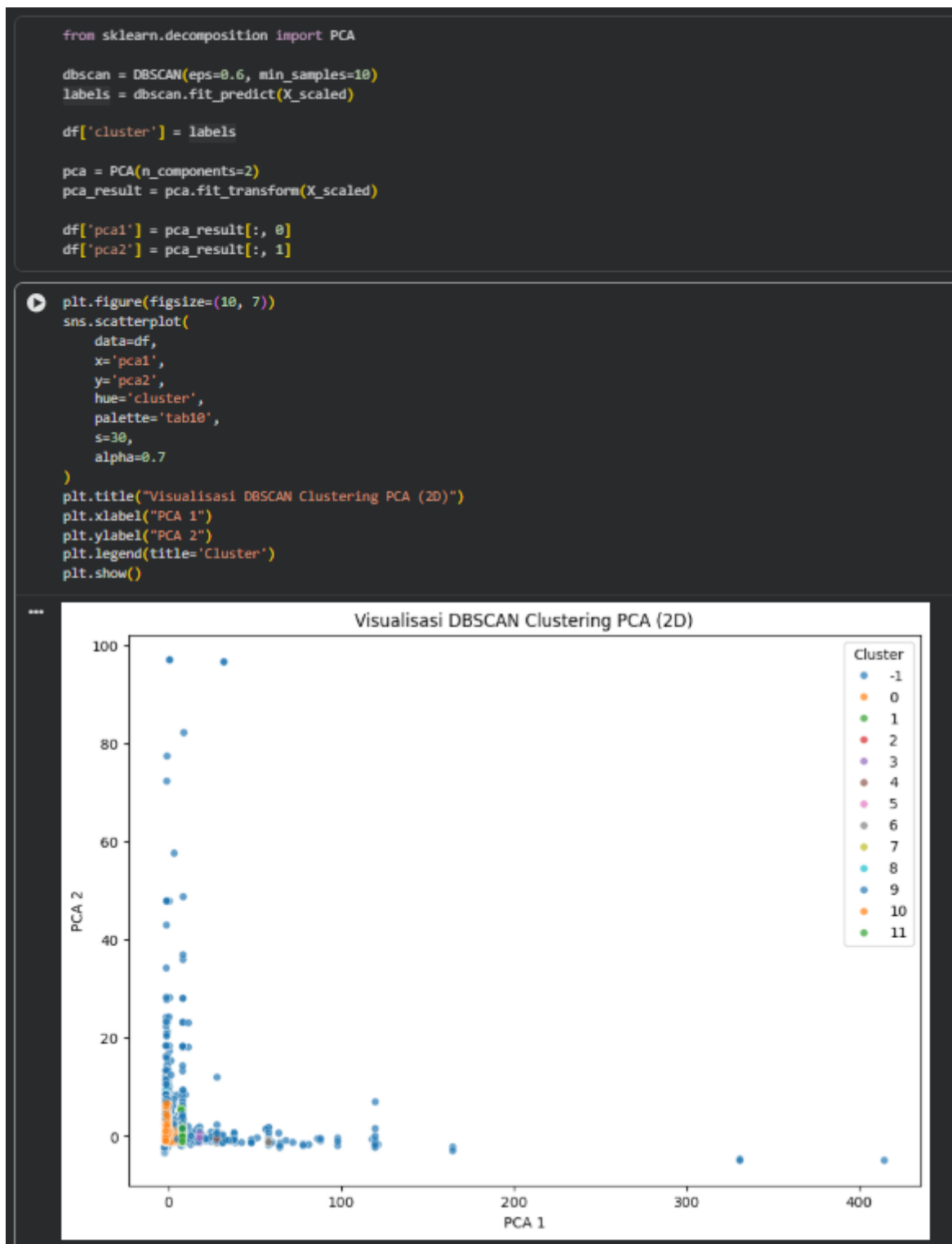
pca = PCA(n_components=2)
pca_result = pca.fit_transform(X_scaled)

df['pca1'] = pca_result[:, 0]
df['pca2'] = pca_result[:, 1]
```

```
plt.figure(figsize=(10, 7))
sns.scatterplot(
    data=df,
    x='pca1',
    y='pca2',
```

```
hue='cluster',
palette='tab10',
s=30,
alpha=0.7
)
plt.title("Visualisasi DBSCAN Clustering PCA (2D)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.legend(title='Cluster')
plt.show()
```

Di sini saya coba $\text{eps}=0.6$ beda dari $\text{eps}=0.30$ sebelumnya untuk lihat pengaruh eps . Menaikkan eps biasanya menyatukan cluster dan jumlah cluster berkurang. Simpan label ke `df['cluster']`. PCA mereduksi dimensi menjadi 2 komponen supaya bisa plot. PCA tidak mempertahankan jarak geografi tetapi membantu visualisasi cluster pada ruang fitur yang rendah dimensi. `sns.scatterplot(... hue='cluster')` memberi warna berbeda per cluster. Outputnya yaitu scatter 2D warna-warni, cluster besar / kecil, titik noise biasanya satu warna (-1) seperti yang ditunjukkan pada Gambar 19.



Gambar 19. Pada bagian ini ditunjukkan Clustering PCA 2D.

r. Visualisasi Cluster pada Peta NYC

```
gdf = gpd.GeoDataFrame(
```



```

df,
geometry=gpd.points_from_xy(df['longitude'], df['latitude']),
crs='EPSG:4326'
)
gdf = gdf.to_crs(epsg=3857)

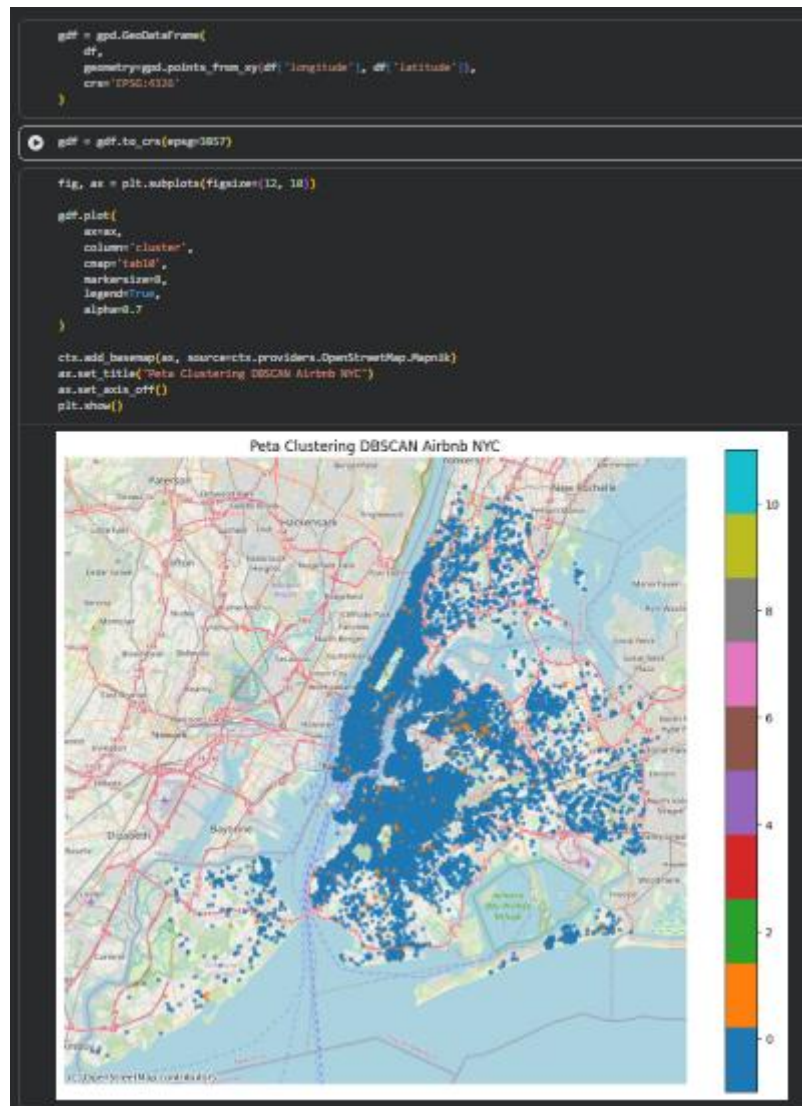
fig, ax = plt.subplots(figsize=(12, 10))

gdf.plot(
    ax=ax,
    column='cluster',
    cmap='tab10',
    markersize=8,
    legend=True,
    alpha=0.7
)

ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)
ax.set_title("Peta Clustering DBSCAN Airbnb NYC")
ax.set_axis_off()
plt.show()

```

Bu blok membuat kembali GeoDataFrame dan mem-plot titik dengan warna column='cluster'. Setiap cluster diberi warna berbeda. ctx.add_basemap menempelkan peta dasar sehingga kita lihat di mana cluster itu berada secara geografis (Manhattan, Brooklyn, dll). legend=True akan menampilkan legend cluster. Jika cluster id banyak (misal > 10), beberapa warna akan diulang. Outputnya adalah peta berwarna dengan cluster; noise biasanya diberi satu warna (-1) dan sering berada di pinggiran seperti yang ditunjukkan pada Gambar 20.



Gambar 20. Pada bagian ini ditunjukkan Cluster pada peta NYC.

3. Interpretasi Hasil Clustering

Berdasarkan DBSCAN yaitu:

1. Jumlah Cluster
 - Cluster 0: sangat besar, berisi ribuan listing
 - Cluster 1: ukuran sedang
 - Cluster 2 dan seterusnya: lebih kecil
 - Noise (-1): lumayan banyak

Jumlah cluster tergantung eps yang dipilih.

2. Noise

Noise cukup banyak karena DBSCAN sangat ketat. Titik-titik yang jaraknya terlalu jauh dari kelompok utama akan langsung dianggap noise. Biasanya ini listing yang jauh dari pusat kota atau berada di pinggiran.

3. Pola Data

- Manhattan sangat padat: masuk cluster besar
- Brooklyn juga padat: masuk cluster lain
- Queens dan Bronx lebih longgar: sebagian masuk noise
- Staten Island banyak yang jadi noise karena listing jarang

Dari peta GPS, cluster terlihat seperti "kumpulan titik" di area tertentu. Titik-titik terpencil otomatis dibuang jadi noise.

4. Kesimpulan dan Hasil Implementasi

Dari seluruh praktikum DBSCAN ini, saya menyimpulkan bahwa algoritma DBSCAN sangat cocok buat data geospasial atau data yang titik-titiknya tersebar di peta. Berbeda dengan K-Means, DBSCAN tidak butuh jumlah cluster di awal, jadi algoritma ini lebih fleksibel. Selain itu, DBSCAN bisa mendeteksi noise dengan baik sehingga titik-titik yang terlalu jauh dari kerumunan tidak ikut ngacauin cluster.

Pada dataset Airbnb NYC, DBSCAN berhasil menemukan pola yang memang sesuai kenyataan: Manhattan dan Brooklyn adalah area paling padat untuk Airbnb. Sementara area seperti Bronx dan Staten Island banyak menghasilkan noise karena listingnya sedikit. Dari visualisasi peta juga terlihat bahwa cluster membentuk area yang memang ramai secara geografis. Praktikum ini membantu saya memahami bahwa menentukan eps dan min_samples itu krusial, dan DBSCAN sangat powerful terutama untuk data koordinat GPS. Saya jadi lebih ngerti bagaimana analisis kepadatan bekerja dan bagaimana pola cluster terbentuk di dunia nyata.

5. Link GitHub (Praktikum dan Latihan Pertemuan 11)

https://github.com/revani18/ML_2025_Revani_3AI01/tree/ba23ca684f5cfa1bfade0539bc82b3ab7fb6c6b8/Praktikum11

Praktikum:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/ba23ca684f5cfa1bfade0539bc82b3ab7fb6c6b8/Praktikum11/Notebooks/praktikum11.ipynb

Latihan:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/ba23ca684f5cfa1bfade0539bc82b3ab7fb6c6b8/Praktikum11/Notebooks/Latihan11_DBSCAN.ipynb